BAKIRÇAY ÜNİVERSİTESİ

2025-2026

VERİTABANI DERSİ FİNAL PROJESİ

ÖĞRENCİ NO

2306010164

AD-SOYAD

Sıla TAŞKIN

ÖĞRENCİ NO

230601019

AD-SOYAD

Zeynep Nur DEĞİRMENÇAY

TARİH

04.01.2026

1. Giriş

1.1. Çalışmanın Genel Tanıtımı

Günümüzde kullanıcılar sosyal medya ve dijital platformlarda gördükleri kıyafet kombinlerini hızlı ve pratik bir şekilde bulmak ve satın almak istemektedir. Ancak bu süreç, farklı platformlar arasında geçiş yapmayı, ürünleri tek tek aramayı ve uyumlu kombin oluşturmayı gerektirdiğinden kullanıcı açısından zaman alıcı ve karmaşık olabilmektedir. Bu proje kapsamında, kullanıcıların bir görsel üzerinden ilham aldıkları kombinleri analiz ederek e-ticaret platformlarındaki ürünlerle eşleştirmeyi ve satın alma sürecini tek bir sistem üzerinden yönetmeyi amaçlayan bütünleşik bir veritabanı tasarımı ele alınmıştır. Çalışmada, farklı sistemlerden gelen verilerin ortak bir veri modeli altında birleştirilmesi hedeflenmiş; kullanıcı, içerik (kombin görseli), ürün kataloğu ve ödeme/faturalandırma verileri tutarlı, ölçeklenebilir ve normalize edilmiş bir yapı içerisinde ele alınmıştır.

Çalışmada, farklı sistemlerden gelen verilerin ortak bir veri modeli altında birleştirilmesi hedeflenmiş; kullanıcı, içerik (kombin görseli), ürün kataloğu ve ödeme/faturalandırma verileri tutarlı, ölçeklenebilir ve normalize edilmiş bir yapı içerisinde ele alınmıştır.

1.2. Seçilen Üçlü Sistemin Kısa Tanıtımı

Projede bütünleştirilen üçlü sistem aşağıdaki bileşenlerden oluşmaktadır:

<u>Sosyal Medya / İlham Sistemi:</u>

Kullanıcıların sosyal medya platformlarında veya dijital ortamlarda gördükleri kıyafet kombinlerini fotoğraf olarak sisteme yükleyebildiği, bu görseller üzerinden kombin parçalarının belirlendiği sistemdir.

<u>E-Ticaret Sistemi (Trendyol Ürün Kataloğu):</u>

Kombin görselinde yer alan parçaların (üst giyim, alt giyim, ayakkabı, aksesuar vb.) Trendyol benzeri bir e-ticaret platformundaki ürünlerle eşleştirildiği; ürünlerin kategori, marka, beden, renk, fiyat ve stok bilgileriyle tutulduğu sistemdir.

<u>Ödeme, Abonelik ve Faturalandırma Sistemi:</u>

Kullanıcıların oluşturdukları kombinleri satın alabildiği, farklı ödeme yöntemleriyle ödeme yapabildiği ve başarılı işlemler sonucunda fatura veya makbuz üretilen sistemdir. Ayrıca gelişmiş özelliklerin (detaylı filtreleme, sınırsız kombin oluşturma, özel kampanyalar) abonelik planları kapsamında sunulması bu sistem içerisinde ele alınmaktadır.

1.3. Neden ve Nasıl Bu Sistemler Bütünleştirilmiştir

Seçilen üçlü sistemin bütünleştirilme nedeni, kullanıcıların kombin keşfetme, ürün bulma ve satın alma süreçlerini tek bir platform üzerinden gerçekleştirme ihtiyacıdır. Sosyal medya platformları ilham kaynağı sağlarken, e-ticaret platformları ürün temini ve satın alma işlevini üstlenmekte; ödeme ve faturalandırma sistemleri ise bu sürecin finansal boyutunu oluşturmaktadır. Bu üç sistemin ayrı ayrı çalışması kullanıcı deneyimini zorlaştırmakta ve veri bütünlüğü açısından sorunlar yaratabilmektedir.

Bu projede, söz konusu sistemler ortak bir veritabanı mimarisi altında birleştirilerek; kullanıcı, kombin, ürün ve finansal işlem verileri arasında anlamlı ilişkiler kurulmuştur. Böylece hem veri tekrarının önüne geçilmiş hem de sistemler arası tutarlılık sağlanmıştır. Tasarlanan bütünleşik yapı, EER modeli ile ifade edilmiş ve daha sonra ilişkisel şemaya dönüştürülerek normalizasyon kurallarına uygun hale getirilmiştir. Üç sistemde ortak olarak bulunan User kavramı tek bir User varlığı altında birleştirilmiştir.

2. Sistemlerin Analizi

2.1. Sistem-1: Sosyal Medya / İlham (Kombin Keşif) Sistemi

Amaç, kullanıcıların kombin fotoğrafları paylaştığı, diğer kullanıcıların kombinleri keşfedebildiği sosyal bir platformdur. Sistem, paylaşılan görsel üzerinde kombini oluşturan parçaları aday öğeler halinde tanımlar (üst giyim, alt giyim, ayakkabı, aksesuar vb.) ve bu aday parçaları daha sonra e-ticaret ürünleriyle eşleştirmeye hazırlar. Sosyal etkileşim (beğeni, yorum, takip) verileri tutulur.

a) Temel varlıklara

- User
- Media
- Post (Kombin Paylaşımı)
- Tag
- PostTag
- Like
- Comment
- PostItem
- Outfit

b) Varlıkların özellikleri

- User (user_id (PK), username, email, password, created_at)
- Post (post_id (PK), caption, post_date)
- Media (media_id (PK), post_id (FK), url, media_type)
- Tag (tag_id (PK), tag_name)
- PostTag (post_id (FK), tag_id (FK))
- Like (like_id (PK), user_id (FK), post_id (FK), like_date)
- Comment (commentid (PK), user_id (FK), post_id (FK), content, created_at)
- PostItem (post_item_id, item_type, color, post_id(FK))
- Outfit (outfit_id, title, description, created_at, user_id(FK))

c) İlişkiler

- User creates Post : 1 – N
- Post receives Media : 1 – N
- Post tagged with Tag : N – N → PostTag
- User likes Post : (User 1–N Like) ve (Post 1–N Like)
- User creates Comment ve Post receives Comment :
- Post contains PostItem : 1 – N
- User creates Outfit : 1 – N

2.2. Sistem-2: Ürün/Katalog & Sepet/Sipariş Sistemi)

Amaç, pazar yeri mantığında ürünlerin listelenmesi; ürün-varyant (beden/renk) yönetimi; satıcıların varyantlara stok ve satış fiyatı tanımlaması; kullanıcının sepete ekleyip sipariş oluşturması ve ödeme/fatura süreçlerine temel oluşturması.

a) Temel varlıklar

- Product
- Category
- Seller (alt türler: IndividualSeller, CorporateSeller)

- ProductVariant
- Stock
- Cart
- CartItem
- Order
- OrderItem

b) Varlıkların özellikleri

- Category(category_id, category_name)
- Seller (seller_id, seller_name, email, rating, seller_type/discriminator)
    - IndividualSeller(seller_id(PK/FK), tc_no)
    - CorporateSeller(seller_id(PK/FK), tax_no)
- Product (product_id, product_name, description, price, category_id(FK), seller_id(FK))
- ProductVariant (variant_id, product_id(FK), color, size, barcode, list_price, sale_price, status)
- Stock (stock_id, variant_id(FK), seller_id(FK), quantity)
- Cart (cart_id, user_id(FK), created_at)
- CartItem (cart_item_id, cart_id(FK), variant_id(FK), quantity, list_price)
- Order (order_id, user_id(FK), order_date, total_amount, order_status)
- OrderItem (order_item_id, order_id(FK), variant_id(FK), quantity, price_at_purchase)

c) İlişkiler

- ProductVariant refers_to SubscriptionPlan
- Product has ProductVariant (Product 1 – N ProductVariant)
- ProductVariant belongs_to Category (Category 1 – N ProductVariant)
- Product belongs_to Seller (Seller 1 – N Product)
- Seller has Stock ve Stock ties Seller–ProductVariant (Seller 1–N Stock, ProductVariant 1–N Stock)
- User has Cart (User 1 – N Cart)
    - Cart contains CartItem (Cart 1 – N CartItem)
    - CartItem is for ProductVariant (ProductVariant 1 – N CartItem)
    - User places Order (User 1 – N Order)
    - Order contains OrderItem (Order 1 – N OrderItem)

2.3. Sistem-3: Kombin Oluşturma + Ödeme/Abonelik/Faturalandırma

Amaç: Sosyal medyada görülen bir kombin postunu, post içindeki aday parçalar (PostItem) üzerinden Trendyol'daki gerçek ÜrünVaryant kayıtlarıyla eşleştirip "kombin sepeti/satın alma"ya dönüştürür. Ayrıca ücretli özellikler (premium öneri, fotoğraftan bulma limiti vb.) abonelik planlarıyla sunulur. Ödeme yöntemi, tutar, tarih, durum ve fatura/makbuz gibi alanlar tutulur.

a) Temel varlıklar

- Outfit
- OutfitItem
- Matching
- SubscriptionPlan
- Subscription
- Payment
- Invoice
- Coupon

b) <u>Varlıkların özellikleri</u>

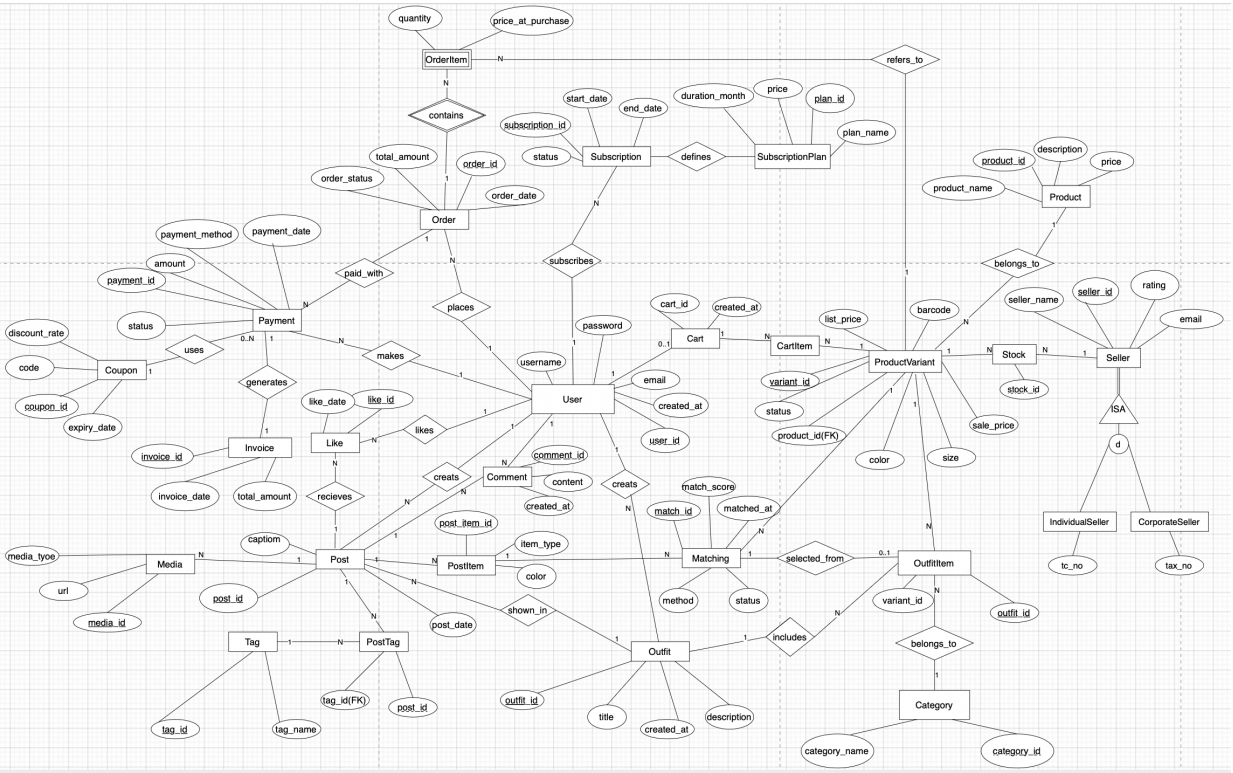
- Outfit(outfit_id, user_id(FK), title, description, created_at)
- OutfitItem(outfit_item_id, outfit_id(FK), variant_id(FK))
- Matching(match_id, match_score, method, status, matched_at, post_item_id(FK), outfit_item_id(FK))
- SubscriptionPlan(plan_id, plan_name, duration_month, price)
- Subscription(subscription_id, user_id(FK), plan_id(FK), start_date, end_date, status)

- Payment(payment_id, amount, payment_method, payment_date, status)
- Invoice(invoice_id, invoice_date, total_amount)
- Coupon(coupon_id, code, discount_rate, expiry_date)

c) <u>İlişkiler</u>

- Outfit includes OutfitItem (1 – N)
- OutfitItem belongs_to ProductVariant (N – 1)
- Matching selected_from OutfitItem
- User creates Outfit (1 – N)
- User subscribes Subscription (1 – N)
- Subscription defines SubscriptionPlan (N – 1)
- User makes Payment (1 – N)
- Payment generates Invoice (1 – 1)
- Coupon uses Payment(1-0..N)
- Order paid_with Payment(1-N)

3. Bütünleştirilmiş ER Diyagramı



EER diyagramından elde edilen tablolar, atomik alanlar kullanılarak **1NF**, kısmi bağımlılıklar giderilerek **2NF**ve transit bağımlılıklar kaldırılarak **3NF** koşullarını sağlayacak şekilde tasarlanmıştır; böylece veri tekrarı ve tutarsızlıklar önlenmiştir.

3.2. Her varlık için kısa açıklamalar

- User: Sistemi kullanan, post paylaşan, kombin oluşturan, abonelik satın alan ve sipariş veren kullanıcıyı temsil eder.
- Post: Kullanıcının paylaştığı kombin içeriğini ve bu içeriğe ait temel açıklama bilgilerini tutar.
- Media: Bir posta ait fotoğraf veya video gibi görsel içerikleri temsil eder.
- Tag: Postların kategorize edilmesi ve keşfedilebilirliğinin artırılması amacıyla kullanılan etiketleri temsil eder.
- PostTag: Post ve Tag varlıkları arasındaki çoktan çoğa ilişkiyi temsil eden ilişki varlığıdır.
- Like: Kullanıcıların postlara verdiği beğeni bilgilerini tutar.
- Comment: Kullanıcıların postlar üzerinde yaptığı yorumları temsil eder.
- PostItem: Bir post üzerinde tespit edilen ve kombini oluşturan aday parçaları (üst giyim, alt giyim, ayakkabı vb.) temsil eder.
- Outfit: Kullanıcının oluşturduğu kombinleri ve bu kombinlere ait açıklama bilgilerini tutar.
- OutfitItem: Bir kombini oluşturan ve seçilen ürün varyantlarını temsil eder.
- Matching: PostItem ile OutfitItem arasında yapılan eşleştirmeleri ve eşleşme skorlarını tutar.
- Product: Satıcı tarafından sisteme eklenen ve satışa sunulan temel ürün bilgisini temsil eder.
- ProductVariant: Ürünün renk, beden gibi farklı varyasyonlarını temsil eder.
- Category: Ürün ve ürün varyantlarının ait olduğu kategorileri temsil eder.
- Seller: Sistemde ürün satan gerçek veya tüzel kişileri temsil eder.
- IndividualSeller: Bireysel olarak satış yapan satıcı türünü temsil eder ve Seller varlığının alt türüdür.
- CorporateSeller: Kurumsal olarak satış yapan satıcı türünü temsil eder ve Seller varlığının alt türüdür.
- Stock: Satıcıya ait ürün varyantlarının stok durumunu temsil eder.
- Cart: Kullanıcının satın almayı planladığı ürün varyantlarını geçici olarak tuttuğu sepeti temsil eder.
- CartItem: Sepet içerisinde yer alan ürün varyantlarını ve miktar bilgilerini temsil eder.
- Order: Kullanıcının sepet üzerinden oluşturduğu satın alma işlemini temsil eder.
- OrderItem: Bir siparişe ait ürün varyantlarını ve satın alma detaylarını temsil eder.
- SubscriptionPlan: Kullanıcılara sunulan ücretli özellik paketlerini ve bu paketlerin kapsamını temsil eder.
- Subscription: Kullanıcının bir abonelik planına yaptığı üyelik bilgisini temsil eder.
- Payment: Kullanıcının sipariş veya abonelik için gerçekleştirdiği ödeme işlemlerini temsil eder.
- Invoice: Gerçekleştirilen ödeme sonucunda oluşturulan fatura bilgisini temsil eder.
- Coupon: Ödeme sırasında kullanılabilen indirim kodlarını temsil eder.

4. ER to EER Açıklamaları

4.1. Hiyerarşi (Specialization/Generalization) eklendi:

Seller → IndividualSeller / CorporateSeller

- **ER'deki durum:** ER modelinde "Seller" tek varlık olarak tutuluyordu.
- **EER'de yapılan değişiklik:** "Seller" varlığı üst varlık (supertype) yapıldı; iki alt tür tanımlandı:
  - **IndividualSeller** (tc_no)
  - **CorporateSeller** (tax_no)
- **Gereksinim / Neden:** Satıcılar iki farklı tipe ayrıldığı için (bireysel/kurumsal), bazı alanlar sadece belirli tipe ait (TC no sadece bireysel, vergi no sadece kurumsal). Bu ayrımı ER'de tek tabloda tutmak NULL alan üretirdi ve veri tutarlılığını düşürürdü.
- **Kısıtlar:**
  - **Disjoint (d):** Bir satıcı aynı anda hem bireysel hem kurumsal olamaz.
  - **Total:**Her seller ya bireyseldir ya da kurumsaldır.

4.2. M:N ilişkiler ara (associative) varlıklarla çözüldü:

Outfit ↔ ProductVariant (OutfitItem)

- **ER'deki durum:** Outfit ile ProductVariant arasında "includes" ilişkisi vardı; çoktan çoğa (M:N).
- **EER'de yapılan değişiklik:** Bu ilişki **OutfitItem** ara varlığına dönüştürüldü.
- **Gereksinim / Neden:** Bir outfit içinde aynı üründen **kaç adet** olduğu gibi ilişkiye ait nitelikler doğrudan M:N ilişkiye konulamadığı için ara varlık gerekir. Ayrıca ilişkisel şemaya geçerken M:N ilişki zaten ara tabloya dönüşür; EER'de bunu netleştirmek modelin doğruluğunu artırır.

Post↔Tag

- **ER'deki durum:** Post ile Tag arasında M:N ilişki vardı.
- **EER'de yapılan değişiklik:** Bu ilişki **PostTag** adlı bir ara varlık oluşturularak çözülmüştür. (post_id,tag_id) Böylece etiketlerin tekrar kullanılabilirliği sağlanmış ve bir postun birden fazla etiketle ilişkilendirilmesi mümkün hale gelmiştir.

Order ↔ ProductVariant

- **ER'deki durum:**Order ile ProductVariant arasında M:N ilişki vardı.
- **EER'de yapılan değişiklik:**Bu ilişki OrderItem adlı ara varlıkla modellenmiştir.(order_id,variant_id,quantity,price_at_purchase)

4.3. Post içindeki parçalar ayrıştırıldı: Post → PostItem

- **ER'deki durum:** ER tarafında "Post" tek varlıktı; kombinin içindeki her parça ayrı tutulmuyordu veya tek yerde kalıyordu.
- **EER'de yapılan değişiklik:** Post'tan bağımsız "**PostItem**" varlığı oluşturuldu (ör. post_item_id, item_type, color).
- **Gereksinim / Neden:** Bir post tek bir üründen oluşmaz; kombinde üst/alt/ayakkabı gibi birden fazla parça olur. "Fotoğraftan bulma / eşleştirme" özelliği için sistemin her parçayı ayrı varlık olarak tutması gerekir; aksi halde eşleştirme tek bir ürüne indirgenir ve işlev eksik kalır.

4.4. Eşleştirme süreci ayrı varlık yapıldı: Matching (PostItem ↔ ProductVariant)

- **ER'deki durum:** "Esleme" gibi bir ilişki/alan düşünülmüştü ama süreç detayları net değildi.
- **EER'de yapılan değişiklik:** "**Matching**" varlığı eklendi ve şu nitelikler modele alındı: match_id, match_score, method(AI/manuel), status, matched_at
- **Gereksinim / Neden:** Aynı PostItem birden fazla ProductVariant ile eşleşebilir (alternatifler), eşleşmenin doğruluk skoru ve yöntemi tutulmalıdır. Ayrıca kullanıcı bazen önerilenler arasından seçim yapar; bu da süreç yönetimi (status) gerektirir. Bu nedenle eşleştirme bir "olay/süreç" olarak bağımsız varlık olmalıdır.

4.5. Sepet ve sipariş yapısı netleştirildi: Cart / CartItem ile Order / OrderItem ayrımı

- **ER'deki durum:** Ürün satın alma akışında "sipariş" vardı ama sepet/sipariş ayrımı belirsiz kalabiliyordu.
- **EER'de yapılan değişiklik:**
  - **Cart** ve **CartItem** (kullanıcının geçici seçimleri)
  - **Order** ve **OrderItem** (satın alma sonrası kalıcı kayıt)
- **Gereksinim / Neden:** Sepet kalıcı sipariş değildir; kullanıcı sepeti boşaltabilir/değiştirebilir. Sipariş oluştuğunda ürün fiyatı "satın alma anındaki fiyat" olarak saklanmalıdır (price_at_purchase). Bu nedenle iki ayrı yapı gereklidir.

4.6. Ödeme ve faturalandırma ayrıştırıldı: Payment → Invoice

- **ER'deki durum:** Ödeme işlemi vardı, fatura bazen ödeme ile aynı şey gibi duruyordu.
- **EER'de yapılan değişiklik:** Payment ile ilişkili ayrı "Invoice" varlığı kuruldu (Payment generates Invoice).
- **Gereksinim / Neden:** Her ödeme fatura üretmeyebilir (başarısız/iptal). Başarılı ödemede fatura kesilir ve kendi numarası/tarihi/toplamı olur. Bu ayrım raporlama ve denetim için şarttır.

4.7. Stock bir "özellik" değil, ilişkiyi temsil eden varlık olarak ele alındı: Seller ↔ ProductVariant

- **ER'deki durum:** Stok bazen Product'ın tek bir attribute'u gibi düşünülebilir.
- **EER'de yapılan değişiklik:** "**Stock**" varlığı oluşturulup Seller–ProductVariant arasında konumlandırıldı.
- **Gereksinim / Neden:** Aynı ProductVariant **farklı seller'larda** farklı stok ve fiyatla satılabilir. Bu yüzden stok, tek bir product attribute'u olamaz; "hangi seller hangi varyantı kaç adet satıyor" bilgisidir.

5. İlişkisel Cebir Sorguları

5.1. Belirli bir kullanıcının oluşturduğu tüm postları listeleme

Amaç: Kullanıcının paylaştığı kombin postlarını görmek.
İlişkisel Cebir:
π post_id, post_date
(σ user_id = 5 (Post))

5.2. Bir postta yer alan tüm parçaları (PostItem) listeleme

Amaç: Bir kombin fotoğrafındaki tüm ürün parçalarını görmek.
İlişkisel Cebir:
π post_item_id, item_type, color
(σ post_id = 10 (PostItem))

5.3. Bir post parçası için eşleşen ürün varyantlarını ve skorlarını getirme

Amaç: Fotoğraftaki parçaya önerilen Trendyol ürünlerini görmek.
İlişkisel Cebir:
π variant_id, match_score, method
(σ post_item_id = 20 (Matching))

5.4. Kullanıcının oluşturduğu outfit'lerde yer alan ürün varyantlarını listeleme

Amaç: Kullanıcının kombinlerinde kullandığı ürünleri görmek.
İlişkisel Cebir:
π outfit_id, variant_id
(σ user_id = 5 (Outfit) ⋈ OutfitItem)

5.5. Bir siparişte satın alınan ürünler ve adetleri

Amaç: Sipariş içeriğini detaylı görmek.
İlişkisel Cebir:
π order_id, variant_id, quantity
(σ order_id = 12 (OrderItem))

5.6. GROUPING 1: Satıcıya göre toplam stok miktarları

Amaç: Her satıcının toplam kaç adet ürünü olduğunu bulmak.
İlişkisel Cebir:
γ seller_id, SUM(quantity) → total_stock
(Stock)

5.7.GROUPING 2: Ürün varyantlarının kaç farklı post parçası ile eşleştiği

Amaç: Hangi ürünün kaç farklı kombin parçasında önerildiğini görmek.

İlişkisel Cebir:

γ variant_id, COUNT(match_id) → match_count

(Matching)

5.8.Kullanıcının yaptığı siparişlerin toplam tutarları

Amaç: Kullanıcı bazında sipariş harcamalarını görmek.

İlişkisel Cebir:

γ user_id, SUM(total_amount) → total_spent

(Order)

5.9.Bir outfit'te kaç ürün olduğu

İlişkisel Cebir:

γ outfit_id, COUNT(variant_id) → item_count

(OutfitItem)

6. Mantıksal Tasarım ve SQL Komutları

6.1.CREATE TABLE komutları

```
CREATE TABLE "User" (
   user_id INT PRIMARY KEY,
   username VARCHAR(50),
   email VARCHAR(100),
   password VARCHAR(100),
   created_at DATE
);
```

Create Table'ların 1 örneği. User tablosunu oluşturma.

6.2.INSERT, UPDATE, DELETE

```
INSERT INTO Post (post_id, user_id, post_date, caption) VALUES
(1, 1, '2025-01-10', 'Günlük kombin'),
(2, 2, '2025-01-11', 'Ofis stili'),
(3, 3, '2025-01-12', 'Kış kombini'),
(4, 4, '2025-01-13', 'Bohem tarz'),
(5, 5, '2025-01-14', 'Sokak stili'),
(6, 6, '2025-01-15', 'Şık akşam yemeği'),
(7, 7, '2025-01-16', 'Rahat kombin'),
(8, 8, '2025-01-17', 'Chill birafta sonu'),
(9, 9, '2025-01-18', 'Spor stil'),
(10,10,'2025-01-19', 'Okul kombini')
```

Insert kodlarının bir örneği. Post tablosuna değer girme işlemi.

```
UPDATE Post
SET caption = 'Chill bir hafta sonu'
WHERE post_id = 8;
```

Update kodlarının bir örneği. Yanlış girilen bir değer update edilmiştir.

```
DELETE FROM Stock
WHERE variant_id = 15;
insert into stock(stock_id,seller_id,variant_id,quantity)values
(15,5,15,20)
```

Delete örneği. Ürün stokta kalmadığı için tablodan silinir sonra güncellenir.

6.2.1.5.Matching

```
INSERT INTO Matching (match_id, post_item_id, variant_id, match_score, method) VALUES
(21, 1, 11, 0.87, 'Renk'),
(22, 1, 12, 0.82, 'Kategori'),

(23, 2, 13, 0.90, 'Renk + Kategori'),
(24, 2, 14, 0.78, 'Yapay Zeka'),

(25, 3, 15, 0.94, 'Görsel Tanıma'),
(26, 3, 16, 0.80, 'Kategori'),

(27, 4, 17, 0.88, 'Yapay Zeka'),
(28, 4, 18, 0.76, 'Renk'),

(29, 5, 19, 0.91, 'Hibrit'),
(30, 5, 20, 0.83, 'Kategori'),

(31, 6, 1,  0.89, 'Renk'),
(32, 6, 2,  0.84, 'Yapay Zeka'),

(33, 7, 3,  0.92, 'Görsel Tanıma'),
(34, 7, 4,  0.79, 'Kategori'),

(35, 8, 5,  0.86, 'Renk'),
(36, 8, 6,  0.81, 'Hibrit'),

(37, 9, 7,  0.90, 'Yapay Zeka'),
(38, 9, 8,  0.88, 'Görsel Tanıma'),

(39, 10, 9, 0.85, 'Kategori'),
(40, 10, 10,0.93, 'Renk + Kategori');
```

Bu sorgu, post öğeleri ile ürün varyantları arasındaki eşleşmeleri, kullanılan yöntem ve eşleşme skoru ile birlikte Matching tablosuna ekler.

6.3. En az 5 SELECT sorgusu

Bazı Select sorgusu örnekleri. Diğer sorguların kodları sql dosyasında mevcut.

6.3.1 Bir kullanıcının oluşturduğu kombinin tüm parçaları

```
SELECT p.post_id, pi.post_item_id, pi.item_type, pi.color
FROM Post p
JOIN PostItem pi ON p.post_id = pi.post_id
        WHERE p.post_id = 3;
```

	post_id integer	post_item_id integer	item_type character varying (50)	color character varying (30)
1	3	5	Üst Giyim	Gri
2	3	6	Alt Giyim	Lacivert

6.3.2. Her bir kombin paylaşımında yer alan parça sayısı

```
SELECT p.post_id, COUNT(pi.post_item_id) AS item_count
FROM Post p
JOIN PostItem pi ON p.post_id = pi.post_id
GROUP BY p.post_id;
```

	post_id [PK] integer	item_count bigint
1	5	2
2	4	1
3	10	2
4	6	2
5	2	2
6	7	1
7	1	2
8	8	2
9	9	2
10	3	2

6.3.3. Kullanıcıların yaptığı siparişlerin toplam tutarı

```
SELECT o.user_id, SUM(o.total_amount) AS total_spent
FROM "Order" o
GROUP BY o.user_id;
```

	user_id integer	total_spent numeric
1	9	899.99
2	3	1499.99
3	5	1849.97
4	4	2149.97
5	10	549.99
6	6	179.99
7	2	1349.98
8	7	2249.98
9	1	2249.96
10	8	299.99

6.3.4. Satıcı bazında toplam stok miktarı

6.3.5. Bir kombin paylaşımındaki parçalara önerilen ürün varyantları listele

6.3.6. En pahalı ilk 5 ürün

6.3.7. Stok sayısı 35'in altında olan ürünler

6.4. En az 3 anlamlı içiçe SELECT sorgusu

6.4.1.Hiç sipariş vermemiş olan kullanıcılar

```
SELECT user_id, username
FROM "User"
WHERE user_id NOT IN (
   SELECT DISTINCT user_id
   FROM "Order"
);
```

	user_id [PK] integer	username character varying (50)

6.4.2. Hiçbir otufitte kullanılmayan ürünler

```
SELECT variant_id
FROM ProductVariant
WHERE variant_id NOT IN (
   SELECT DISTINCT variant_id
   FROM OutfitItem
);
```

6.4.3. En çok harcama yapan kullanıcı

```
SELECT user_id, total_amount
FROM "Order"
WHERE total_amount = (
   SELECT MAX(total_amount)
   FROM "Order"
```

);

	user_id integer	total_amount numeric (10,2)
1	7	2249.98

6.4.4. En az 1 siparişte satılmış olan ürün varyantları

```
SELECT variant_id, price
FROM ProductVariant
WHERE variant_id IN (
   SELECT DISTINCT variant_id
   FROM OrderItem
);
```

6.5. Trigger

Trigger 1: Sipariş eklendiğinde stok düşürme

```
CREATE TRIGGER trg_reduce_stock
AFTER INSERT ON OrderItem
FOR EACH ROW

EXECUTE FUNCTION reduce_stock_after_order();
```

Trigger 2: Negatif stok engelleme

```
CREATE TRIGGER trg_prevent_negative_stock
BEFORE UPDATE ON Stock
FOR EACH ROW

EXECUTE FUNCTION prevent_negative_stock();
```

Trigger 3: Sipariş içeriği değişince toplam tutarı güncelleme

```
CREATE TRIGGER trg_update_order_total_after_insert
AFTER INSERT ON OrderItem
FOR EACH ROW

EXECUTE FUNCTION update_order_total();
```

6.6. View

Kullanıcı Bazında Toplam Harcama Raporu

```
CREATE VIEW vw_user_total_spending AS
SELECT
   u.user_id,
   u.username,
   SUM(o.total_amount) AS total_spent
FROM "User" u
JOIN "Order" o ON u.user_id = o.user_id
GROUP BY u.user_id, u.username;

SELECT * FROM vw_user_total_spending;
```

Sipariş Detayları Raporu

```
CREATE OR REPLACE VIEW vw_order_details AS
SELECT
   o.order_id,
   o.user_id,
```

```
    oi.variant_id,
    oi.quantity,
    oi.price_at_purchase,
    (oi.quantity * oi.price_at_purchase) AS line_total
FROM "Order" o
JOIN OrderItem oi ON o.order_id = oi.order_id;

SELECT * FROM vw_order_details
WHERE order_id = 1;

SELECT * FROM vw_order_details
WHERE order_id = 10;

SELECT * FROM vw_order_details
WHERE order_id = 2;
```

Ürün Stok Durumu Raporu

```
CREATE OR REPLACE VIEW vw_stock_status AS
SELECT
    s.variant_id,
    s.quantity,
    CASE
        WHEN s.quantity < 5 THEN 'Kritik'
        WHEN s.quantity BETWEEN 5 AND 20 THEN 'Orta'
        ELSE 'Yeterli'
    END AS stock_status
FROM Stock s;

SELECT * FROM vw_stock_status;

CREATE OR REPLACE VIEW vw_matching_method_performance AS
SELECT
    method,
    AVG(match_score) AS avg_score,
    COUNT(*) AS usage_count
FROM Matching
GROUP BY method;
SELECT *
FROM vw_matching_method_performance
ORDER BY avg_score DESC;
```

Ürün eşleştirme yöntemlerinin ortalama başarı skorları analiz edilmiştir.

6.7. Stored Procedure

Bir tane Stored Procedure örneği diğer ikisinin kodları sql dosyasında mevcut.

- Kullanıcıya Ait Siparişleri Listeleme

```
CREATE OR REPLACE FUNCTION fn_get_user_orders(p_user_id INT)
RETURNS TABLE (
    order_id INT,
    order_date DATE,
    total_amount DECIMAL
)
LANGUAGE plpgsql
```

```
AS $$
BEGIN
   RETURN QUERY
   SELECT
      o.order_id,
      o.order_date,
      o.total_amount
   FROM "Order" o
   WHERE o.user_id = p_user_id;
END;
$$;
```

Çalıştırma:

```
SELECT * FROM fn_get_user_orders(1);
```

- Sipariş Detaylarını Getirme
- Kritik Stok Seviyesindeki Ürünleri Getirme

7. Veri Girişi (Sample Data)

7.1.1. Temel Varlıklar İçin Veri Girişi

Sistemde yer alan kullanıcılar, satıcılar ve ürünler için örnek kayıtlar oluşturulmuştur. Ürünler farklı kategorilerde tanımlanmış ve her ürün için birden fazla varyant oluşturularak fiyat bilgileri girilmiştir. Bu sayede ürün–varyant ilişkisi test edilmiştir.Ürün ve varyantlara ait örnek veri girişleri önceki bölümlerde verilen INSERT komutları ile gerçekleştirilmiştir.

7.1.2. İlişkisel Tablolar İçin Veri Girişi

Ürün stokları, satıcı–ürün ilişkisini gösterecek şekilde tanımlanmış ve her ürün varyantı için stok miktarları girilmiştir. Böylece stok yönetimi, stok güncelleme ve negatif stok kontrolü gibi iş kurallarının çalışması test edilmiştir.Stok verileri, senaryoya uygun miktarlarda olacak şekilde sisteme eklenmiştir.

7.1.3. Sosyal Paylaşım ve Kombin Verileri

Kullanıcıların kombin paylaşımlarını temsil eden postlar ve bu postlara ait parça bilgileri sisteme eklenmiştir. Bir kombin paylaşımında birden fazla parça yer alabilmesi sağlanarak post–postItem ilişkisi test edilmiştir. Bu veriler sayesinde kombin parçalarına yönelik ürün eşleştirme ve öneri mekanizmaları doğrulanmıştır.

7.1.4. Sipariş ve Satın Alma Senaryoları

Kullanıcıların ürün satın alma işlemlerini temsil eden siparişler ve sipariş içerikleri sisteme eklenmiştir. Siparişler aracılığıyla stok düşürme, stok geri yükleme ve sipariş toplam tutarının otomatik güncellenmesi gibi tetikleyicilerin doğru çalıştığı gözlemlenmiştir.
Sipariş verileri farklı kullanıcılar ve ürünler kullanılarak oluşturulmuştur.

7.1.5. Öneri Mekanizması Verileri

Kombin parçaları ile ürün varyantları arasında oluşturulan eşleştirme kayıtları sayesinde, öneri mekanizmasının veri modeli test edilmiştir. Eşleştirme skorları ve kullanılan yöntemler, raporlama ve analiz işlemlerinde kullanılmak üzere sisteme eklenmiştir.

7.2. 3Anlamlı örnek senaryolarla veri girişi

Senaryo 1: Kullanıcının Kombin Paylaşımı ve Ürün Önerisi

Bir kullanıcı, oluşturduğu günlük kombinini sistem üzerinde paylaşmaktadır. Bu paylaşım sırasında kombin birden fazla parça (üst giyim, alt giyim, ayakkabı vb.) içermektedir. Her bir parça için sistem, uygun ürün varyantlarını eşleştirerek kullanıcıya öneriler sunmaktadır. Bu senaryo kapsamında: Post ve PostItem tablolarına veri girilmiştir. Kombin parçaları ile ürün varyantları arasında Matching kayıtları oluşturulmuştur.Eşleştirme yöntemleri ve skorları tanımlanmıştır.

Senaryo 2: Kullanıcının Ürün Satın Alma Süreci
Kullanıcı, sistemde önerilen ürünler arasından bir veya daha fazla ürünü satın almaktadır. Satın alma işlemi sırasında bir sipariş oluşturulmakta ve siparişe ait ürünler OrderItem tablosuna eklenmektedir. Bu senaryo kapsamında: Order ve OrderItem tablolarına veri girilmiştir. Siparişe ürün eklendiğinde stok miktarı otomatik olarak azaltılmıştır. Sipariş toplam tutarı tetikleyiciler aracılığıyla otomatik olarak güncellenmiştir.

Senaryo 3: Sipariş İptali ve Stok Geri Yükleme
Kullanıcının daha önce oluşturduğu bir siparişin iptal edilmesi durumunda, siparişe ait ürünler sistemden silinmekte ve ilgili ürünlerin stok miktarları otomatik olarak geri yüklenmektedir.Bu senaryo kapsamında: OrderItem tablosundan silme işlemi gerçekleştirilmiştir. Stok geri yükleme tetikleyicisi devreye girerek stok tutarlılığı korunmuştur.

Senaryo 4: Stok Yönetimi ve Negatif Stok Kontrolü
Sistemde yer alan ürünlerin stok miktarlarının geçersiz değerlere düşmesini engellemek amacıyla negatif stok kontrolü uygulanmaktadır. Stok miktarının sıfırın altına düşmeye çalışılması durumunda işlem tetikleyici tarafından engellenmektedir. Bu senaryo kapsamında: Stock tablosu üzerinde yapılan güncellemeler test edilmiştir. Negatif stok engelleyen tetikleyici devreye girmiştir.

Senaryo 5: Raporlama ve Analiz Süreci
Sisteme girilen veriler kullanılarak kullanıcı harcamaları, stok durumları ve ürün eşleştirme performansları analiz edilmiştir. Bu analizler, oluşturulan görünümler (view) ve saklı yordamlar aracılığıyla gerçekleştirilmiştir.

8. Sonuç ve Değerlendirme

Bu çalışmada, kombin paylaşımı ve ürün öneri süreçlerini destekleyen bir veritabanı sistemi tasarlanmış ve gerçekleştirilmiştir. Tasarlanan EER diyagramı doğrultusunda mantıksal ve fiziksel veritabanı yapıları oluşturulmuş, tablolar arası ilişkiler tanımlanmıştır.
Veritabanı üzerinde oluşturulan sorgular, görünümler (view), tetikleyiciler (trigger) ve saklı yordamlar (stored procedure/function) sayesinde veri bütünlüğü korunmuş ve raporlama işlemleri etkin bir şekilde gerçekleştirilmiştir. Örnek veri girişleri ve senaryolar ile sistemin gerçek hayata uygunluğu test edilmiştir.
Sonuç olarak, geliştirilen veritabanı modeli hem fonksiyonel gereksinimleri karşılamakta hem de genişletilebilir bir yapı sunmaktadır.

8.1. Zorluklar ve öneriler

Bu çalışmanın gerçekleştirilmesi sürecinde, özellikle karmaşık tablo ilişkilerinin modellenmesi ve veri bütünlüğünün sağlanması aşamalarında bazı zorluklarla karşılaşılmıştır. Zayıf varlıkların tanımlanması, tetikleyicilerin doğru sırada ve doğru koşullarla çalıştırılması, dikkat gerektiren konular arasında yer almıştır. Bununla birlikte, PostgreSQL ortamında saklı yordamlar ve tetikleyiciler kullanılarak bu zorluklar aşılmıştır.
İlerleyen çalışmalarda, öneri mekanizmasının daha gelişmiş algoritmalarla desteklenmesi, performans optimizasyonlarının yapılması ve kullanıcı etkileşimlerini artıracak ek raporlama bileşenlerinin sisteme entegre edilmesi önerilmektedir.